Software Design Document

Study Buddy
Social Media Platform

Version 2.0
March 15, 2026

Team: *Cutie Coders*
Matthieu Resurreccion, Jason Hernandez-Velazquez,
Jose Michael Monge, Juan Cruz,
Matthew Berleson, Angie Raymundo Alas

Supervisor: Edmund Dantes
Computer Science Department
California State University, Northridge

Table of Contents

Table of Contents 2

1. Introduction 3
- 1.1. Purpose 3
- 1.2. Scope 3
- 1.3. Overview 3
- 1.4. Reference Material 3
- 1.5. Definitions and Acronyms 3

2. System Overview 4

3. System Architecture 4
- 3.1. Architectural Design 4
- 3.2. Interface Design 4
- 3.3. Decomposition Description 5
- 3.4. Design Rationale 5

4. Component Design/Detailed Design 5
- 4.1. Client-Side Design 5
- 4.2. Server-Side Design 6
- 4.3. Machine Learning Design 7

5. Human Interface Design 7
- 5.1. Overview of User Interface 7
- 5.2. Screen Images 8
- 5.3. Screen Objects and Actions 8

6. Requirements Matrix 9

7. Appendices 10

1. Introduction

1.1 Purpose

This software design document's purpose is to describe the architecture and system design of the Study Buddy platform. The document provides an overarching overview of the system's components that will guide the implementation of the project. The document will continue to evolve with the project based on the design decisions, constraints, or oversights. This will ensure a maintainable, reliable shared understanding of the Study Buddy platform. This document's intended audience is the Cutie Coders group and Professor Dantes.

1.2 Scope

Study Buddy aims to increase productivity in students and bring them together in an effortless manner. The application provides a space where students can easily connect and form study groups. They would then be able to share resources about their chosen topic. Its purpose also includes combating student loneliness by expanding the reach of academic collaboration beyond the classroom. The web application supports educational success and fosters a sense of belonging among students. Initially, the application would be available only to CSUN students for their classes taken at the university. The release will prioritize a reliable and user-friendly platform for students to share their interests and career goals.

1.3 Overview

This document describes how the Study Buddy platform is structured so that CSUN students are successful in joining study groups.

1.4 Reference Material

Meetup.com

Meetup is a platform that connects individuals with similar interests to find, join, and host group events over a shared activity. It aims to foster real-life relationships by offering people a chance to gain a sense of belonging.

Vampr

Vampr is a social media platform that connects musicians with a desire to collaborate with other musicians to create projects, play concerts, or have a jam session. It utilizes a similar swiping mechanism that dating apps are notable for.

1.5 Definitions and Acronyms

2. System Overview

Study Buddy is a social media app that will be built upon the FARM (FastAPI, React, MongoDB) tech stack. As a FARM app generally goes, the frontend will be designed as a collective of React components (which supports all modern-day browsers), the backend server developed using FastAPI, and data stored in a MongoDB server.

The frontend will consist of the necessary pages that altogether comprise the core Study Buddy app, such as a login page, profile page, and chat box interface. Within each page are reusable React components that reoccur throughout the platform, like a sidebar, loading screen, and menu items. Each of these elements will act as child routes to a parent route that serves as the fundamental layout of the website.

As for backend development, FastAPI allows us to develop a REST API to facilitate data between the user and the database using standard HTTP methods (POST, GET, PATCH, DELETE). Endpoints will be set up for retrieving and requesting information like post, user, and group data. This information will be stored in a MongoDB database and contextualized by their defined schemas.

3. System Architecture

3.1 Architectural Design

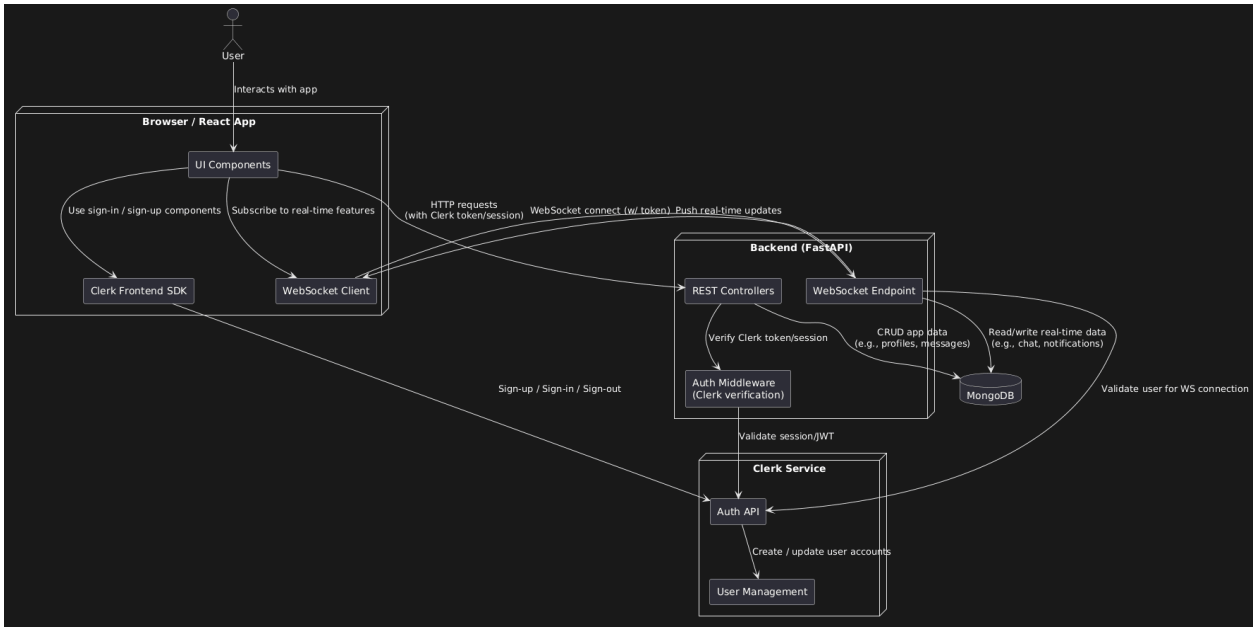


Figure 3.1.1 System Architectural Design Diagram

3.2 Interface Design

As any web application generally goes, the frontend communicates with the backend through a set of RESTful endpoints that support profile management, posts, commenting, liking, and messaging. User registration and login are handled through Clerk, which provides session tokens for authorization. The backend interfaces with the database through schemas for users, posts, comments, groups, and messages.

3.3 Decomposition Description

The frontend can be broken down into the following parts. Firstly, the React app confirms that a user is authenticated with a session token provided by Clerk. If false, then the login screen is displayed. Else, a general layout page is shown to the user. This layout checks if the user credentials are properly loaded or a loading component takes place. When the user is properly instantiated, the layout is presented as a sidebar component aside a child page, like the user's feed, chat box, or profile, that includes its own components, each demonstrating their own unique functionalities. For example, the feed page will consist of a component delegated to creating and viewing story posts alongside another sidebar component.

The backend is structured as such. All API endpoints and request handlers are located in ~/routers. Database queries and CRUD operations occur in ~/repos. Schemas for users and groups happen in ~/models. Configurations like environment variables, dependencies, and database connections are in ~/core.

3.4 Design Rationale

The design of this project is standard for a React web-based application, such that the frontend pages are a host of reusable components, features, assets, and HTTP requests.

The same can be said about the backend. However, amidst the beginning of development came the decision to change the stack from MERN (MongoDB, Express.js, React, Node.js) to FARM (FastAPI, React, MongoDB). The choice was made in favor of FastAPI streamlining backend functions that Express.js requires developers to integrate manually. It is also faster, stronger typed, and more suitable for machine learning integration, since we aim to feature a profile-based matchmaking system. On the downside, it doesn't integrate real-time systems as well as Node.js does, but this flaw is not much of a concern considering the application is merely a no-budget, student-led senior-project, not a large-scale product meant for the public's use.

4. Component Design/Detailed Design

4.1 Client-Side Design

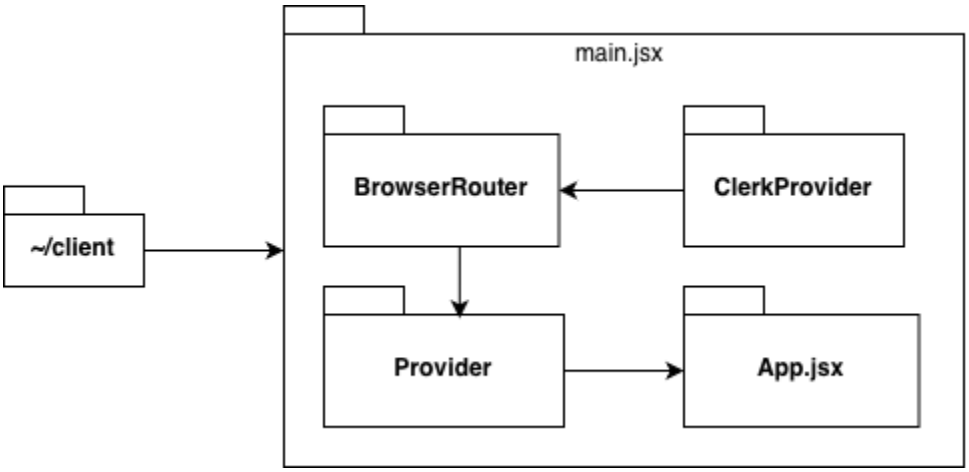


Figure 4.1.1 Client-Side Package Diagram

ClerkProvider handles user authentication and passes down user information and sign-in state to the rest of the app. BrowserRouter and Provider pass down additional contexts like routes, UI states, and session flags.

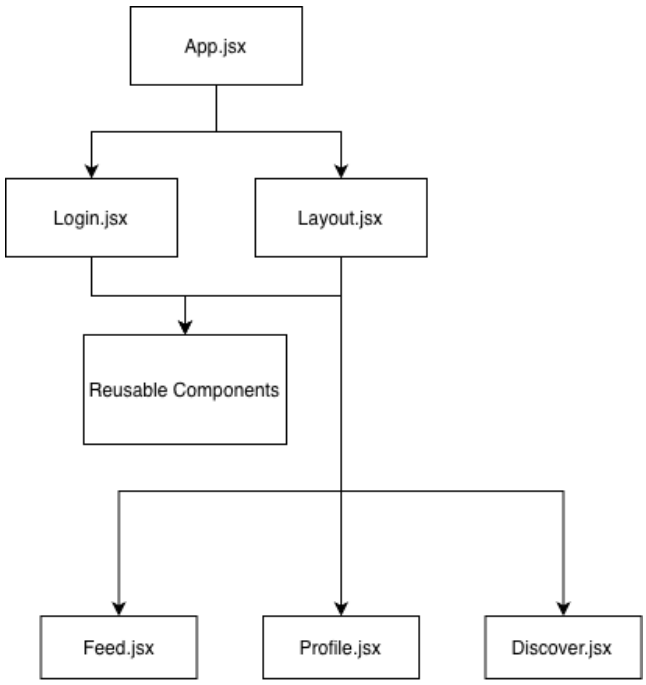


Figure 4.1.2 App.jsx Component Diagram

If the user is not yet authenticated, then App.jsx displays Login.jsx, otherwise it shows the main app, which is wrapped in Layout.jsx and whichever route the user has selected. Each of these React pages reuse components throughout the codebase.

4.2 Server-Side Design

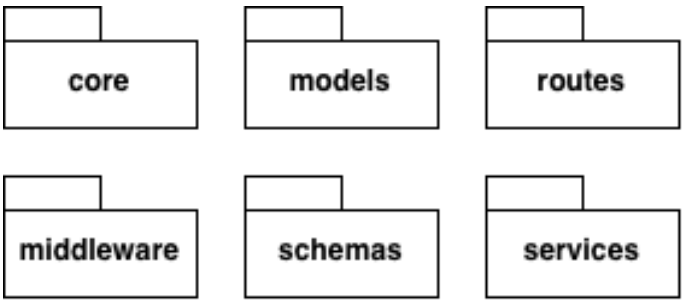


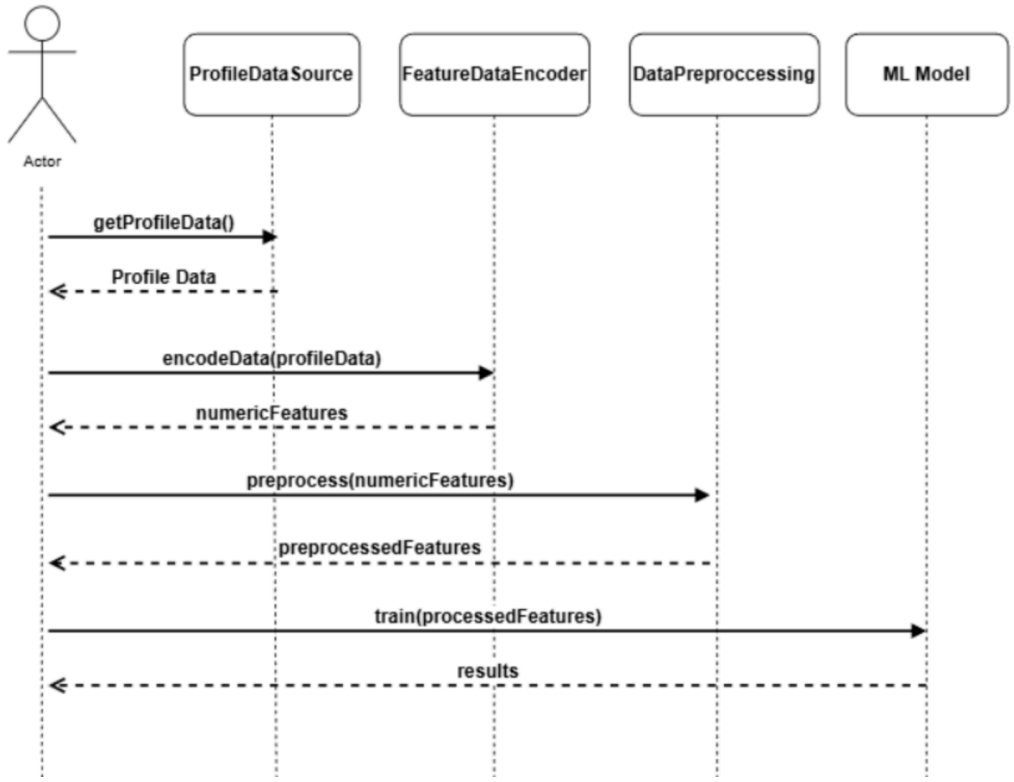
Figure 4.2.1 Backend Package Diagram

The proposed backend will consist of the following directories:

- ~/core: Configurations
- ~/routers: API endpoints
- ~/models: Database methods
- ~/schemas: Pydantic models
- ~/middleware: Clerk authorization integration

- ~/services: Controller logic used by routers

4.3 Machine Learning Design



4.3.1 Profile-Based Matchmaking ML Pipeline Sequence Diagram

Above is a proposed pipeline for a profile-based matchmaking system using ML that the project aspired to feature.

5. Human Interface Design

5.1 Overview of User Interface

The user interface should be reminiscent of a traditional social media application with the following pages:

1. Login/Registration
2. Feed
3. Discover
4. Messages
5. Profile
6. Connections

However, the true functionality of the application is the emphasis on forming and participating in study groups, providing academic resources, encouraging real-life peer connection, and providing a means to academic conversation. With these goals in mind, some additional pages bear the essence of the project's mission:

1. Study Groups
2. Student Center
3. Events
4. Forum

5.2 Screen Images

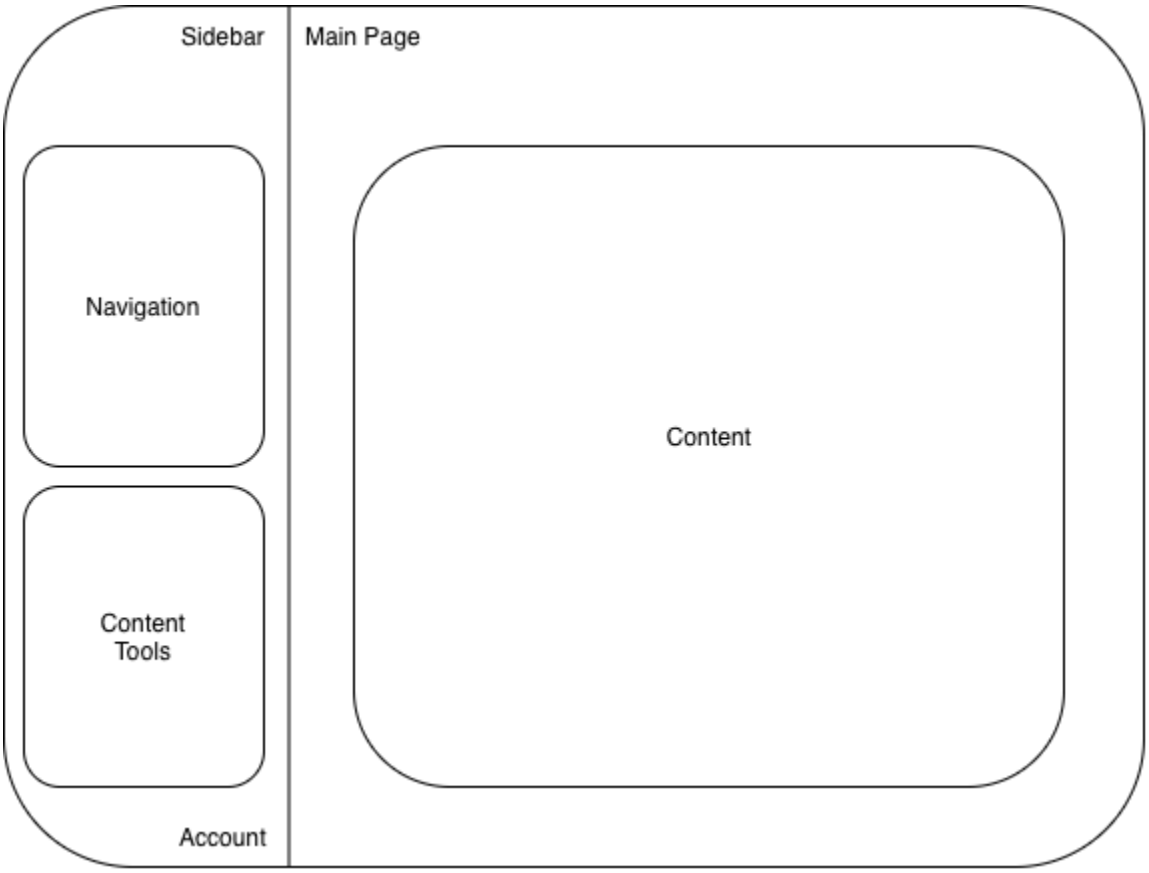


Figure 5.2.1 General Layout

For the GUI design of the login, feed, discover, messages, profile, and connections, refer to Instagram and LinkedIn. For study groups and events refer to Facebook. For forum, refer to Reddit. For student services, refer to library.csun.edu's 'Quicklinks'.

5.3 Screen Objects and Actions

Figure 5.3.1 General Object Card Layout

A majority of the screen objects, like group links, user profiles, and posts will follow this same card template.

6. Requirements Matrix

Requirement	Description	Satisfaction Component/Proof
GR_SRS_001	The platform shall allow users to register and authenticate profiles.	Figure 3.1.1
GR_SRS_002	The platform shall securely store user data using hashing methods.	Figure 3.1.1
GR_SRS_003	The platform shall allow users to chat in real time.	Section 5.1
GR_SRS_004	The platform shall recommend and match users to each other based on profile similarity.	Figure 4.3.1
GR_SRS_005	The platform shall allow users to form and join study groups.	Section 5.1
GR_SRS_006	The platform shall integrate Clerk authentication	Figure 3.1.1
GR_SRS_007	The platform shall be supported on all major platforms (Chrome, Firefox, Safari, etc).	Section 2
GR_SRS_008	The platform shall store profile, study group, chat, and other data in a MongoDB database.	Figure 3.1.1
GR_SRS_0	The platform shall allow users to	Figure 3.1.1

09	register and authenticate profiles.	
GR_SRS_001	The platform shall securely store user data using hashing methods.	Figure 3.1.1
GR_SRS_002	The platform shall allow users to chat in real time.	Figure 3.1.1
GR_SRS_003	The platform shall recommend and match users to each other based on profile similarity.	Figure 4.3.1
GR_SRS_004	The platform shall allow users to form and join study groups.	Section 5.1

7. Appendices